

Expressoes regulares do lexer

Este documento lista as expressoes regulares equivalentes as regras lexico-sintaticas implementadas em src/lexer.rs.

Observacao importante:

- O lexer atual e um automato finito (nao usa crate de regex).
- Algumas regras aceitam tokens incompletos no fim do arquivo (ex.: string sem fechar).
- As regex abaixo representam o comportamento reconhecido pelo codigo atual.

Classes basicas

- Letra ASCII: [A-Za-z]
- Dígito: [0-9]
- Espaço: [\t\r\n]
- Espacos (1+): [\t\r\n]+

Identificadores e palavras-chave

- Identificador: `^[A-Za-z_][A-Za-z0-9_]*$`
- Palavra-chave (lista atual):

`^(?:if|else|while|for|return|int|float|char|void|class|struct|enum|const|static|public|private|protected|virtual|override|abstract|template|typedef|namespace|using|include|long|short|signed|unsigned|do|switch|case|break|continue|goto|default|bool|double|new|delete|sizeof|and|)|$`

Literais numericos

- Inteiro: `^[0-9]+$`
- Float (como o lexer aceita): `^[0-9]+\.[0-9]*$`

Nota:

- O lexer aceita numero com ponto final sem digitos depois (ex.: 12.).

Literais de string e char

- String fechada (forma canonica): `^(?:"[^\"]")*$`
- String aceita pelo lexer (fecha opcional no EOF): `^(?:"[^\"]")*"?$`
- Char fechado (forma canonica): `^(?:\"[^\"]\")$`
- Char aceito pelo lexer (fecha opcional em alguns cenarios): `^(?:"[^\"]")?$`

Comentarios

- Comentario de linha: `*/[/[^\n]]*$`
- Comentario de bloco fechado: `/*{[^\s\S]}+*/$`
- Comentario de bloco aceito no EOF sem fechar: `/*{[^\s\S]}+$`

Operadores

- Incremento: `^++$`
- Mais igual: `^+= $`
- Adicao: `^+$`
- Decremento: `^~$`
- Menos igual: `^-= $`
- Seta: `^>$`
- Subtracao: `^-$`
- Vezes igual: `^*= $`
- Multiplicacao: `^^$`
- Divisao igual: `^/= $`
- Divisao: `^/$`
- Mod igual: `^%= $`
- Modulo: `^%$`
- Igualdade: `^==$`
- Atribuicao: `^=$`
- Menor igual: `^<=$`
- Shift esquerda: `^<<$`
- Menor: `^<$`
- Maior igual: `^>=$`
- Shift direita: `^>>$`
- Maior: `^>$`
- Diferente: `^!= $`
- Not logico: `^!$`
- And logico: `^&&$`
- Bit and: `^&$`
- Or logico: `^||$`
- Bit or: `^|$`
- Escopo: `^::$`
- Dois pontos: `^:$`
- Bit xor: `^^$`
- Bit not: `^~$`
- Interrogacao: `^?$`

Separadores e pontuacao

- Ponto e virgula: `^;$`
- Virgula: `^,$`
- Abre parenteses: `^{($`
- Fecha parenteses: `^)$`
- Abre chave: `^{{$`
- Fecha chave: `^}{$`
- Abre colchete: `^{[$`
- Fecha colchete: `^]$`
- Ponto membro: `^.$`

Ordem pratica de reconhecimento

Para evitar ambiguidades, em qualquer implementacao via regex use preferencia por tokens mais longos antes dos curtos, por exemplo:

1. ++, +=, +
2. --, -=, >, -
3. ==, =
4. <=, <, <
5. ||, =, >, >
6. !=, !
7. &&, &
8. ||, |
9. ::, :